RESUMEN SISTEMAS OPERATIVOS

1. INTRODUCCIÓN

1.1 ¿QUÉ ES UN SISTEMA OPERATIVO?

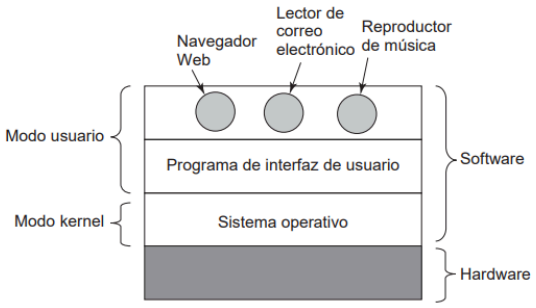


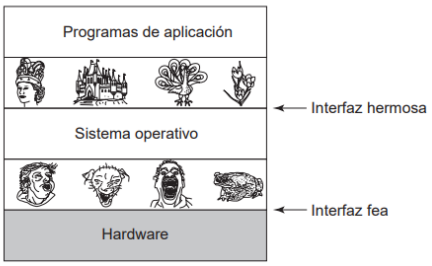
Figura 1-1. Ubicación del sistema operativo.

Es difícil definir qué es un sistema operativo aparte de decir que es el **software** que se ejecuta en **modo kernel** (además de que esto no siempre es cierto). Parte del problema es que los sistemas operativos realizan dos funciones básicas que no están relacionadas:

- **El sistema operativo como una máquina extendida**: La arquitectura de la mayoría de las computadoras a nivel de lenguaje máquina es primitiva y compleja de programar.

  La **abstracción** es la clave para lidiar con la **complejidad**. Una abstracción que casi cualquier usuario de computadora comprende es el archivo: es una pieza útil de información, como una fotografía digital, un mensaje de correo electrónico almacenado o una página Web. Es más fácil lidiar con fotografías, correos electrónicos y páginas Web que con los detalles de los discos.

El trabajo del sistema operativo es crear buenas abstracciones para después implementar y administrar los objetos abstractos entonces creados



- **El sistema operativo como administrador de recursos**:La administración de recursos incluye el **multiplexaje** (compartir) de recursos en dos formas distintas: en el **tiempo** y en el **espacio**.

En la perspectiva alterna, el trabajo del sistema operativo es proporcionar una **asignación ordenada y controlada** de los procesadores, memorias y dispositivos de E/S, **entre los diversos programas que compiten por estos recursos.**

Cuando un recurso se **multiplexa** en el **tiempo**, los distintos programas o usuarios **toman turnos para utilizarlo.**

El otro tipo de **multiplexaje** es en el **espacio**. En vez de que los clientes tomen turnos, cada uno obtiene **una parte del recurso.**

1.3 REVISIÓN DEL HARDWARE DE COMPUTADORA

La CPU, la memoria y los dispositivos de E/S están conectados mediante un **bus del**

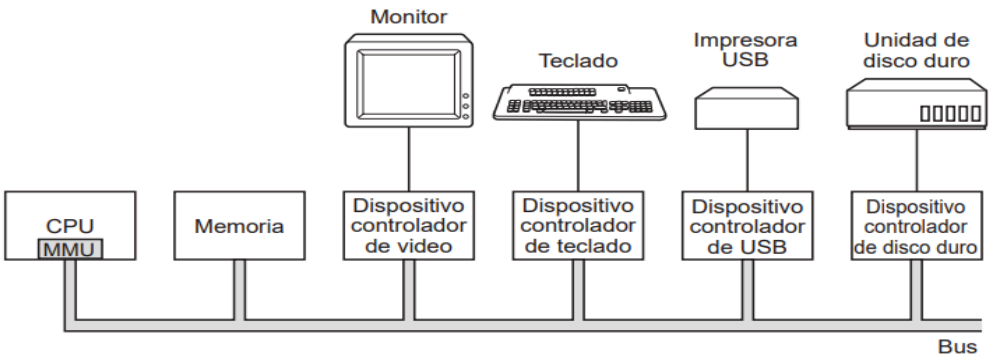


Figura 1-6. Algunos de los componentes de una computadora personal simple.

sistema y se comunican entre sí a través de este bus:

1.3.1 Procesadores

El "**cerebro**" de la computadora es la **CPU**, que obtiene las **instrucciones de la memoria** y las **ejecuta.** El ciclo básico de toda CPU es obtener la primera instrucción de memoria, decodificarla para determinar su tipo y operandos, ejecutarla y después obtener, decodificar y ejecutar las instrucciones subsiguientes. El ciclo se repite hasta que el programa termina.

Además de los registros generales utilizados para contener variables y resultados temporales, la mayoría de las computadoras tienen varios registros especiales que están visibles para el programador.

- **Contador de programa (Program Counter):** Contiene la dirección de memoria de la siguiente instrucción a obtener; se actualiza automáticamente para apuntar a la siguiente.
- **Apuntador de pila (Stack Pointer):** Apunta a la parte superior de la pila actual en memoria, la cual guarda los valores de los procedimientos activos (entrados pero no salidos).
- **PSW (Palabra de estado del programa):** Registro que contiene los bits de código de condición.

El sistema operativo debe estar al tanto de todos los registros.

La mayoría de las CPU, con excepción de las extremadamente simples que se utilizan en los sistemas integrados, tienen **dos modos: modo kernel y modo usuario**. Por lo general, un bit en el PSW controla el modo. Al operar en **modo kernel**, la CPU puede ejecutar **cualquier instrucció**n de su conjunto de instrucciones y utilizar **todas** las **características** del **hardware**. El sistema operativo opera en modo kernel, lo cual le da acceso al hardware completo.

En contraste, los **programas de usuario** operan en modo de usuario, el cual les permite ejecutar sólo un **subconjunto** de las **instrucciones** y les da acceso sólo a un subconjunto de las características.

Para obtener **servicios del sistema operativo**, un programa usuario debe lanzar una **llamada al sistema** (system call), la cual se atrapa en el kernel e invoca al sistema operativo. La instrucción **TRAP** cambia del **modo usuario** al **modo kernel** e inicia el sistema operativo.

Chips con multihilamiento y multinúcleo

Multihilamiento o **hiperhilamiento:**. Para una primera aproximación, lo que hace es permitir que la CPU contenga el estado de **dos hilos de ejecución** (threads) distintos y luego alterne entre uno y otro con una escala de tiempo en nanosegundos.

Memoria

Una memoria debe ser más rápida que la velocidad de ejecución de una instrucción, de manera que la memoria no detenga a la CPU.

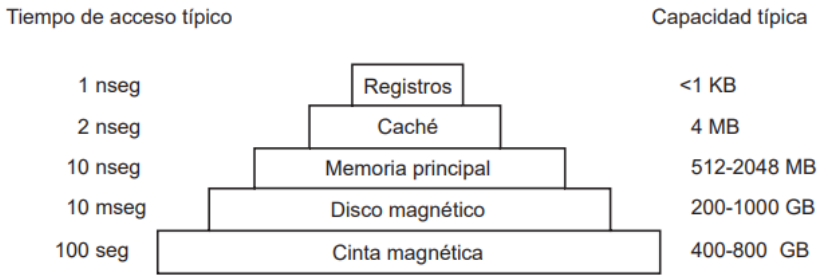


Figura 1-9. Una común jerarquía de memoria. Los números son sólo aproximaciones.

Registros: Compuestos del mismo material que la CPU y, por ende, tienen la misma rapidez.

Memoria caché: El hardware la controla de **manera parcial**. La memoria principal se divide en **líneas de caché**. Las líneas de caché que se utilizan con más frecuencia se mantienen en una caché de **alta velocidad,** ubicada **dentro** o muy **cerca** de la **CPU**

- Cuando el programa necesita leer una palabra de memoria, el hardware de la caché comprueba si la línea que se requiere se encuentra en la caché. Si es así (a lo cual se le conoce como **acierto de caché**) la petición de la caché **se cumple** y **no se envía una petición** de memoria a través del bus hacia la memoria principal.
- La memoria caché está **limitada en tamaño** debido a su alto costo.
- Las cachés son una idea tan útil que las CPUs modernas tienen dos de ellas. La **caché L1** o de primer nivel está siempre dentro de la CPU, y por lo

general alimenta las instrucciones decodificadas al motor de ejecución de la CPU.

- **Caché L2**, que contiene varios megabytes de palabras de memoria utilizadas recientemente. La diferencia entre las cachés L1 y L2 está en la velocidad. El acceso a la caché L1 se realiza sin ningún retraso, mientras que el acceso a la caché L2 requiere un retraso de uno o dos ciclos de reloj.

Memoria Principal:

También conocida como **RAM** (Random Access Memory, Memoria de Acceso Aleatorio).

- Además de la memoria principal, muchas computadoras tienen una pequeña cantidad de **memoria de acceso aleatorio no volátil**. A diferencia de la RAM, la memoria no volátil no pierde su contenido cuando se desconecta la energía.
  - La **ROM** (Read Only Memory, Memoria de sólo lectura) se programa en la fábrica y no puede modificarse después. Es rápida y económica.
  - La **EEPROM** (Electrically Erasable PROM, PROM eléctricamente borrable) y la memoria flash son también no volátiles, pero en contraste con la ROM se pueden borrar y volver a escribir datos en ellas.
  - **CMOS**, la cual es volátil. Muchas computadoras utilizan memoria CMOS para guardar la fecha y hora actuales

Discos

Un disco consiste en uno o más **platos** que giran a 5400, 7200 o 10,800 rpm. Un **brazo mecánico**, con un **punto de giro** colocado en una esquina, se mueve sobre los platos de manera similar al brazo de la aguja en un viejo tocadiscos.

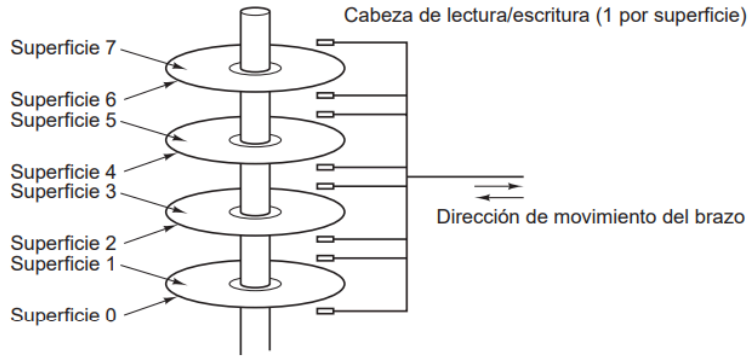


Figura 1-10. Estructura de una unidad de disco.

En cualquier posición dada del brazo, cada una de las cabezas puede leer una región anular conocida como **pista** (track). En conjunto, todas las pistas para una posición dada del brazo forman un **cilindro** (cylinder).

Muchas computadoras presentan un esquema conocido como **memoria virtual**, el cual permite la ejecución de **programas más grandes** que la **memoria física** al colocarlos en el **disco** y utilizar la memoria principal como un tipo de **caché** para las partes que se ejecutan con **más frecuencia**. Este esquema requiere la **reasignación de direcciones de memoria al instante**, para convertir la dirección que el programa generó en la dirección física en la RAM

en donde se encuentra la palabra. Esta asignación se realiza mediante una parte de la CPU conocida como **MMU** (Memory Management Unit, Unidad de Administración de Memoria)

Cintas

La última capa de la jerarquía en la memoria es la cinta magnética. Este medio se utiliza con frecuencia como r**espaldo para el almacenamiento en disco** y para contener conjuntos de datos muy extensos.

Dispositivos de E/S

Los dispositivos de E/S generalmente constan de dos partes: un dispositivo **controlador** y el **dispositivo en sí.** El dispositivo controlador es un **chip** o conjunto de chips que **controla físicamente** el **dispositivo**. Por ejemplo, acepta los comandos del sistema operativo para leer datos del dispositivo y los lleva a cabo.

Los dispositivos tienen **interfaces bastante simples**, debido a que no pueden hacer mucho y también para estandarizarlas. Esto último es necesario de manera que cualquier dispositivo controlador de disco IDE (Integrated Drive Electronics) significa Electrónica de unidades integradas.

El software que se comunica con un dispositivo controlador, que le proporciona comandos y acepta respuestas, se conoce como **driver** (controlador).Para utilizar el driver, se tiene que colocar en el sistema operativo de manera que pueda ejecutarse en **modo kernel**.

Las operaciones de entrada y salida se pueden realizar de tres maneras distintas.

1. **Un programa de usuario emite una llamada al sistema**, que el kernel posteriormente **traduce** en una llamada al procedimiento para el **driver apropiado**. Después el driver inicia la E/S y permanece en un ciclo estrecho, **sondeando** en forma continua al dispositivo para ver si ha terminado. Una vez terminada la E/S, el driver coloca los datos (si los hay) en donde se necesitan y regresa. Después el sistema operativo devuelve el control al llamador. A este método se le conoce como espera ocupada y tiene la desventaja de que mantiene ocupada la CPU sondeando al dispositivo hasta que éste termina.
2. El segundo método consiste en que el **driver inicie** el **dispositivo** y le **pida generar** una **interrupción** cuando **termine**. En este punto el driver regresa. Luego, el sistema operativo bloquea el programa llamador si es necesario y busca otro trabajo por hacer. Cuando el dispositivo controlador detecta el final de la transferencia, genera una interrupción para indicar que la operación se ha completado

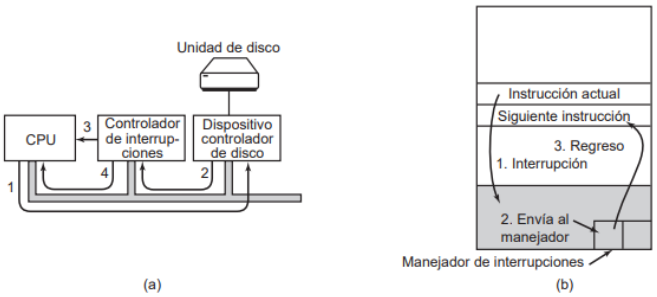


Figura 1-11. (a) Los pasos para iniciar un dispositivo de E/S y obtener una interrupción. (b) El procesamiento de interrupciones involucra tomar la interrupción, ejecutar el manejador de interrupciones y regresar al programa de usuario.

3. El tercer método para realizar operaciones de E/S hace uso de un **chip especial** llamado **DMA** (Direct Memory Access; Acceso directo a memoria) que puede controlar el **flujo de bits** entre la memoria y un dispositivo controlador sin la intervención constante de la CPU. La CPU configura el chip DMA, le indica cuántos bytes debe transferir, las direcciones de dispositivo y de memoria involucradas, la instrucción y deja que haga su trabajo. Cuando el chip DMA termina genera una interrupción, la cual se maneja de la manera antes descrita

Buses

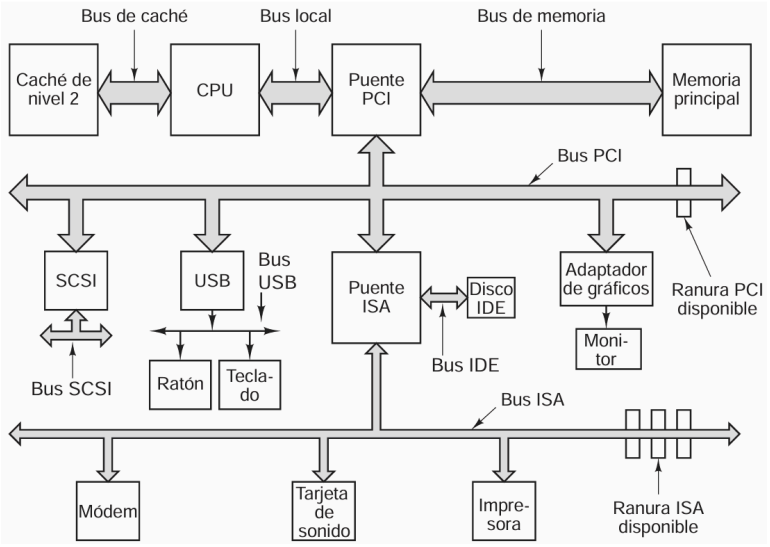


Figura 1-12. La estructura de un sistema Pentium extenso.

Los dos **buses principales** son el bus **ISA** (Industry Standard Architecture, Arquitectura estándar de la industria) de la IBM PC original y su sucesor, el bus **PCI** (Peripheral Component Interconnect, Interconexión de componentes periféricos).

El **USB** (Universal Serial Bus; Bus serial universal) se inventó para conectar a la computadora todos los **dispositivos de E/S lentos**, como el teclado y el ratón.

El bus **SCSI** (Small Computer System Interface, Interfaz para sistemas de cómputo pequeños) es un **bus de alto rendimiento**, diseñado para discos, escáneres y otros dispositivos veloces que necesitan de un ancho de banda considerable.

El sistema operativo tiene que saber qué dispositivos periféricos están conectados a la computadora y cómo configurarlos. Este requerimiento condujo a Intel y Microsoft a diseñar un sistema de PC conocido como **plug and play**. El sistema operativo tiene que saber qué dispositivos periféricos están conectados a la computadora y cómo configurarlos. Este requerimiento condujo a Intel y Microsoft a diseñar un sistema de PC conocido como plug and play. Antes de plug and play, cada tarjeta de E/S tenía un nivel de petición de interrupción fijo y direcciones fijas para sus registros de E/S. Lo cual presentaba un problema cuando dos dispositivos utilizaban la misma interrupción. La función de plug and play es permitir que el sistema recolecte automáticamente la información acerca de los dispositivos de E/S, asigne los niveles de interrupción y las direcciones de E/S de manera central, para que después indique a cada tarjeta cuáles son sus números.

Arranque de la computadora

En forma muy breve, el proceso de arranque del Pentium es el siguiente. Cada Pentium contiene una tarjeta madre (motherboard). En la tarjeta madre o padre hay un programa conocido como **BIOS** (Basic Input Output System, Sistema básico de entrada y salida) del sistema. El BIOS contiene software de E/S de bajo nivel, incluyendo procedimientos para leer el teclado, escribir en la pantalla y realizar operaciones de E/S de disco, entre otras cosas. Hoy en día está contenido en una RAM tipo flash que es no volátil pero el sistema operativo puede actualizarla cuando se encuentran errores en el BIOS.

Cuando se arranca la computadora, el BIOS inicia su ejecución. Primero hace **pruebas** para ver cuánta RAM hay instalada y si el teclado junto con otros dispositivos básicos están instalados y responden en forma correcta. Empieza explorando los **buses** ISA y PCI para detectar todos los dispositivos conectados a ellos. Comúnmente, algunos de estos dispositivos son heredados (es decir, se diseñaron antes de inventar la tecnología plug and play), además de tener valores fijos para los niveles de interrupciones y las direcciones de E/S (que posiblemente se establecen mediante interruptores o puentes en la tarjeta de E/S, pero que el sistema operativo no puede modificar). Estos dispositivos se registran; y los dispositivos plug and play también. Si los dispositivos presentes son distintos de los que había cuando el sistema se inició por última vez, se configuran los nuevos dispositivos

1.5 CONCEPTOS DE LOS SISTEMAS OPERATIVOS

Proceso: Un proceso es en esencia **un programa en ejecución.** Cada proceso tiene asociado un **espacio de direcciones**, una **lista de ubicaciones de memoria** que va desde algún mínimo (generalmente 0) hasta cierto valor máximo, donde el proceso puede leer y escribir información. El espacio de direcciones contiene el **programa ejecutable**, los **datos** del programa y su pila. También hay asociado a cada proceso un conjunto de recursos, que comúnmente incluye registros (el contador de programa y el apuntador de pila, entre ellos), una lista de archivos abiertos, alarmas pendientes, listas de procesos relacionados y toda la demás información necesaria para ejecutar el programa. En esencia, un proceso es un recipiente que guarda toda la información necesaria para ejecutar un programa

Las **llamadas al sistema** de administración de procesos clave son las que se encargan de la creación y la terminación de los procesos. Considere un ejemplo común. Un proceso llamado intérprete de comandos o shell lee comandos de una terminal. El usuario acaba de

escribir un comando, solicitando la compilación de un programa. El shell debe entonces crear un proceso para ejecutar el compilador. Cuando ese proceso ha terminado la compilación, ejecuta una llamada al sistema para terminarse a sí mismo.

Si un proceso puede crear uno o más procesos aparte (conocidos como procesos hijos) y estos procesos a su vez pueden crear procesos hijos, llegamos rápidamente la estructura de **árbol de procesos.**

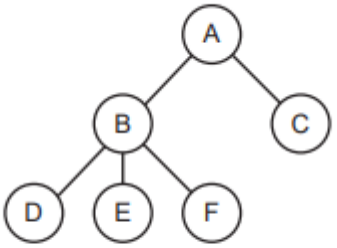


Figura 1-13. Un árbol de proceso. El proceso *A* creó dos proceso hijos, *B* y *C*. El proceso *B* creó tres procesos hijos, *D*, *E* y *F*.

Espacios de direcciones: Cada computadora tiene cierta memoria principal que utiliza para mantener los programas en ejecución. En un sistema operativo muy simple sólo hay un programa a la vez en la memoria.

Archivos:Existen dos tipos de archivos especiales: archivos especiales de bloque y archivos especiales de carácter.

Entrada/Salida:

Protección
El shell
Memoria virtual

1.6 Llamadas al Sistema

• **Definición:** Son la interfaz que permite a los programas de usuario solicitar servicios al sistema operativo.

• **Funcionamiento:** Aunque parecen llamadas a procedimientos normales, utilizan una instrucción **TRAP** para cambiar la CPU del modo usuario al modo kernel y transferir el control al sistema operativo para ejecutar la tarea.

• **POSIX (UNIX):** Se relacionan directamente con los procedimientos de biblioteca (relación uno a uno). Incluyen llamadas para la administración de procesos (`fork`, `exec`), archivos (`read`, `write`) y directorios.

• **Win32 API (Windows):** Es una interfaz con miles de funciones que está **desacoplada** de las verdaderas llamadas al sistema. Son procedimientos de biblioteca que pueden o no invocar al kernel, lo que permite a Microsoft cambiar el kernel sin afectar la compatibilidad de los programas.

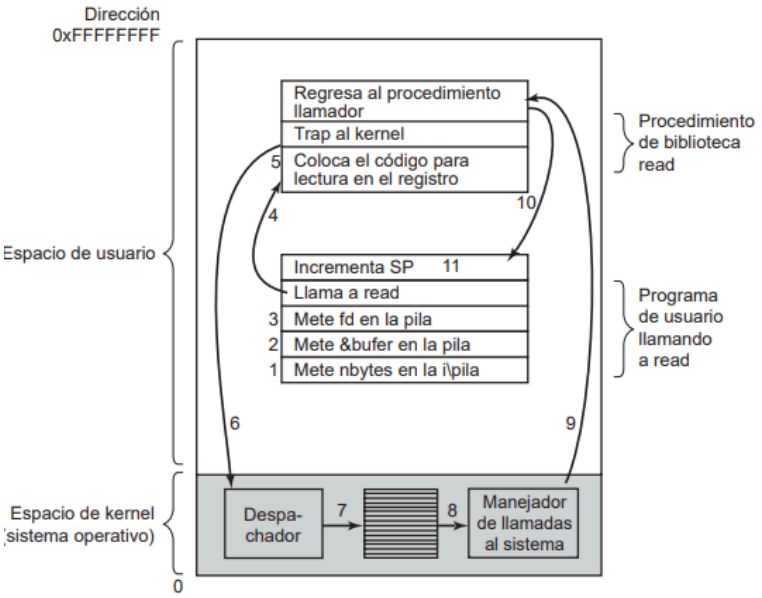


Figura 1-17. Los 11 pasos para realizar la llamada al sistema `read(fd, bufer, nbytes)`.

UNIX	Win32	Descripción
fork	CreateProcess	Crea un nuevo proceso
waitpid	WaitForSingleObject	Puede esperar a que un proceso termine
execve	(ninguno)	CreateProces = fork + execve
exit	ExitProcess	Termina la ejecución
open	CreateFile	Crea un archivo o abre uno existente
close	CloseHandle	Cierra un archivo
read	ReadFile	Lee datos de un archivo
write	WriteFile	Escribe datos en un archivo
lseek	SetFilePointer	Desplaza el apuntador del archivo
stat	GetFileAttributesEx	Obtiene varios atributos de un archivo
mkdir	CreateDirectory	Crea un nuevo directorio
rmdir	RemoveDirectory	Elimina un directorio vacío
link	(ninguno)	Win32 no soporta los enlaces
unlink	DeleteFile	Destruye un archivo existente
mount	(ninguno)	Win32 no soporta el montaje
umount	(ninguno)	Win32 no soporta el montaje
chdir	SetCurrentDirectory	Cambia el directorio de trabajo actual
chmod	(ninguno)	Win32 no soporta la seguridad (aunque NT sí)
kill	(ninguno)	Win32 no soporta las señales
time	GetLocalTime	Obtiene la hora actual

Figura 1-23. Las llamadas a la API Win32 que apenas corresponden a las llamadas de UNIX de la figura 1-18.

1.7 Estructura de un sistema operativo

- • **Sistemas monolíticos:** Es la organización más común. Todo el sistema operativo se ejecuta como **un solo programa en modo kernel**. El sistema operativo se escribe como una **colección de procedimientos**, enlazados entre sí en un solo programa binario

ejecutable extenso. Su estructura básica consta de un programa principal que invoca **procedimientos de servicio**, los cuales a su vez se ayudan de **procedimientos utilitarios**,

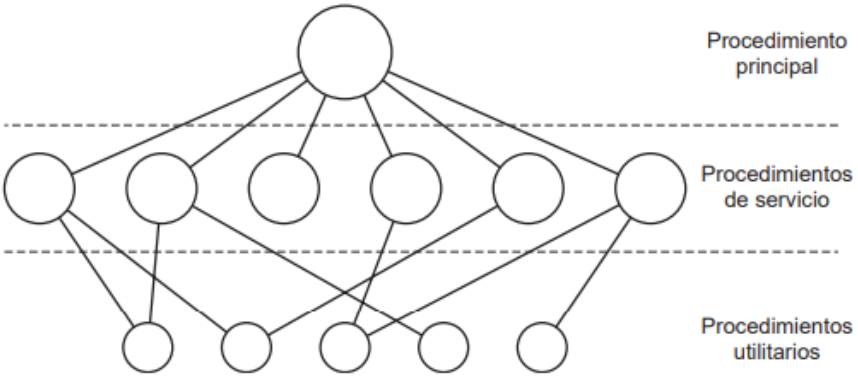


Figura 1-24. Un modelo de estructuración simple para un sistema monolítico.

- • **Sistemas de capas:** El sistema operativo se organiza como una **jerarquía de capas**, cada una construida encima de la que tiene abajo. El primer ejemplo fue el **sistema THE**, que tenía seis capas (desde la asignación del procesador hasta el operador). Una generalización de esto fue el sistema MULTICS, descrito como una serie de **anillos concéntricos**,,.
- • **Microkernels:** Su idea básica es lograr una **alta confiabilidad** al dividir el sistema operativo en **módulos pequeños y bien definidos**. Sólo uno de ellos (el **microkernel**) se ejecuta en modo kernel y el resto se ejecuta como **procesos de usuario** ordinarios sin poder relativamente (como los drivers de dispositivos). Un ejemplo es **MINIX 3**. A menudo se busca colocar el **mecanismo** en el kernel, pero dejar la **directiva** (política) en los procesos de usuario,,.
- **Modelo cliente-servidor:** Es una ligera variación de la idea del microkernel que diferencia dos clases de procesos: los **servidores** (proporcionan cierto servicio) y los **clientes** (utilizan estos servicios). La comunicación se lleva a cabo comúnmente mediante el **paso de mensajes**.
- **Máquinas virtuales:**
    - **Monitor de máquina virtual (VM/370):** Separa las funciones de multiprogramación y de máquina extendida. El monitor se ejecuta en el hardware y realiza la multiprogramación, proporcionando varias **máquinas virtuales** a la siguiente capa, las cuales son **copias exactas** del hardware real.
    - **Hipervisores:** El monitor de máquina virtual tradicional se conoce como **hipervisor de tipo 1** (se ejecuta en el hardware). Un **hipervisor de tipo 2** se ejecuta como un programa de usuario (por ejemplo, en Windows o Linux) e "interpreta" el conjunto de instrucciones,.
    - **JVM:** La Máquina virtual de Java es una arquitectura de computadora que utiliza un intérprete para ejecutar código byte.
    - **Exokernels:** Es una estrategia para **particionar** la máquina, proporcionando a cada usuario un subconjunto de los recursos. En la capa inferior (modo kernel) se ejecuta el **exokernel**, cuyo trabajo es asignar recursos a las máquinas virtuales y comprobar los intentos de utilizarlos.

UNIDAD 2 - PROCESOS E HILOS

El concepto más importante en cualquier sistema operativo es el de **proceso, una abstracción de un programa en ejecución.**
Los procesos son una de las abstracciones más antiguas e importantes que proporcionan los sistemas operativos: proporcionan la capacidad de operar (pseudo) concurrentemente, incluso cuando hay sólo una CPU disponible. Convierten una CPU en varias CPU virtuales.

2.1 PROCESOS

Todas las computadoras modernas ofrecen **varias cosas al mismo tiempo.** En cualquier sistema de multiprogramación, la CPU **conmuta de un proceso a otro con rapidez,** ejecutando cada uno durante décimas o centésimas de milisegundos: hablando en sentido estricto, en cualquier instante la CPU está ejecutando sólo un proceso, y en el transcurso de 1 segundo podría trabajar en varios de ellos, dando la apariencia de un **paralelismo** (o pseudoparalelismo, para distinguirlo del verdadero paralelismo de hardware de los sistemas multiprocesadores con dos o más CPUs que comparten la misma memoria física).
Un proceso no es más que una instancia de un programa en ejecución, incluyendo los valores actuales del contador de programa, los registros y las variables. Esta **conmutación** rápida de un **proceso a otro** se conoce como **multiprogramación**.

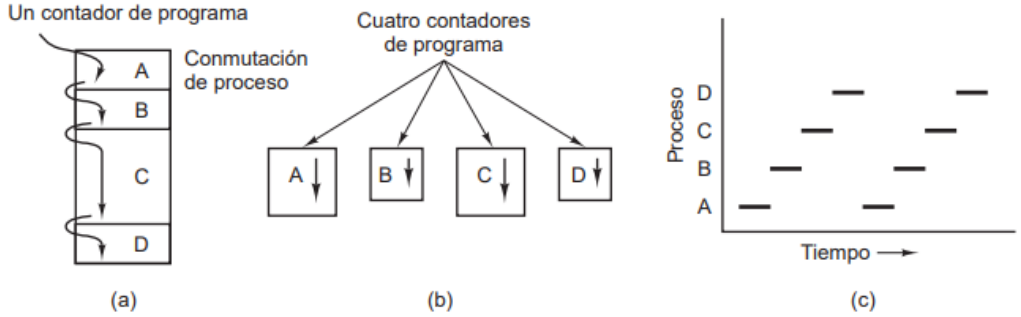


Figura 2-1. (a) Multiprogramación de cuatro programas. (b) Modelo conceptual de cuatro procesos secuenciales independientes. (c) Sólo hay un programa activo a la vez.

La **diferencia** entre un proceso y un programa es sutil pero crucial. La idea clave es que un **proceso es una actividad de cierto tipo:** tiene un programa, una entrada, una salida y un estado. Varios procesos pueden compartir un solo procesador mediante el uso de un algoritmo de planificación.

Creación de un proceso

Hay cuatro eventos principales que provocan la creación de procesos:

1. El arranque del sistema.
2. La ejecución, desde un proceso, de una llamada al sistema para creación de procesos.
3. Una petición de usuario para crear un proceso.
4. El inicio de un trabajo por lotes.

Demonios: Los procesos que permanecen en segundo plano para manejar ciertas actividades como correo electrónico, páginas Web, noticias, impresiones, etcétera. En Unix podemos usar ps para listar los procesos en ejecución.

Además de los procesos que se crean al arranque, posteriormente se pueden crear otros. A menudo, un **proceso en ejecución** emitirá **llamadas al sistema** para **crear** uno o más